

Graph Database Solution for Higher Order Spatial Statistics in the Era of Big Data

CRISTIANO G. SABIU¹

¹*Department of Physics, University of Seoul, Seoul 02504, Republic of Korea*

ABSTRACT

We present GRAMSCI (GRAPh Made Statistics for Cosmological Information), an algorithm and open-source Fortran code for the fast computation of the general N -point spatial correlation function of any discrete point set embedded in $\mathbb{R}^n$. Using KD-trees and a graph database, we count all possible N -tuples in binned configurations within a given length scale. Benchmarks demonstrate the computational advantage over traditional methods; all 3-point configurations up to and beyond the Baryon Acoustic Oscillation scale can be computed on current SDSS data in reasonable time, and we present the first measurements of the 4-point correlation function of ~ 0.5 million SDSS galaxies at $0.43 < z < 0.7$.

Version 2 update. We describe two additions to the original code. First, we replace the binary-search inner loop — which cost $O(m \log m)$ per hub-spoke pair, where m is the mean neighbour count — with a *merge-walk*: a two-pointer sorted-set intersection that exploits the pre-sorted adjacency lists and reduces the inner loop to $O(m)$. For the 3PCF the speedup reaches $2\times$ at $r_{\max} = 55 h^{-1} \text{Mpc}$; for the 4PCF, which contained two binary searches in the k3 inner loop, the speedup reaches $3.8\times$ at $r_{\max} = 40 h^{-1} \text{Mpc}$ — making BAO-scale 4PCF measurements on survey-sized catalogues routine. Results are numerically identical to machine precision. Second, we implement a parity-decomposed 4PCF that separates the signal into even and odd channels, enabling direct tests of parity violation in the galaxy distribution. Third, we provide a Python interface so the Fortran engine can be called directly from NumPy arrays. The code is available at <https://github.com/csabiu/Gramsci>.

Keywords: cosmology: large-scale structure — methods: statistical — methods: numerical — galaxies: statistics

1. INTRODUCTION

The spatial distribution of galaxies encodes information about the primordial density field and its subsequent gravitational evolution. The two-point correlation function (2PCF) has been the workhorse of this programme (e.g. Anderson et al. 2014), but higher-order statistics carry complementary information that is inaccessible to the power spectrum. The 3-point correlation function (3PCF) probes the bispectrum, which constrains primordial non-Gaussianity and galaxy bias (Sefusatti et al. 2006; Gaztañaga & Scoccimarro 2005). The connected 4-point correlation function (4PCF) is related to the primordial trispectrum and provides additional leverage on inflationary models (Sefusatti & Komatsu 2007).

Computing N -point functions from large catalogues is expensive. A brute-force approach requires $O(N^k)$ operations for a k -point function. Tree-based methods reduce this to $O(Nm^{k-1})$, where m is the mean number of neighbours within the maximum scale $r_{\max}$, but the m^{k-1} factor still grows rapidly.

Several codes are publicly available. TREECORR (Jarvis et al. 2004) provides optimised 2PCF and 3PCF estimators for projected and three-dimensional catalogues. CORRFUNC (Sinha & Garrison 2020) uses SIMD vectorisation (AVX2/AVX-512) for maximum 2PCF throughput. ENCORE (Philcox & Slepian 2022) expands the N -PCF in a spherical-harmonic basis, replacing direct enumeration with matrix products; this achieves favourable scaling at high neighbour density but does not directly yield a configuration-space connected 4PCF with parity decomposition.

GRAMSCI (Sabiou et al. 2019) uses a graph-database approach: it builds a compact sorted adjacency struc-

ture once from a KD-tree, then queries it for any combination of 2-, 3-, and 4-point statistics without revisiting the original coordinates. This paper extends the original work in three directions (Section 3): an $O(m)$ merge-walk inner kernel (Section 3.1), a parity-decomposed 4PCF (Section 3.2), and a Python interface (Section 3.3).

2. THE GRAMSCI ALGORITHM

We summarise the algorithm here; full details are in Sabiu et al. (2019).

2.1. Graph construction

Given a catalogue of N objects, GRAMSCI builds a KD-tree and finds, for each object i , all neighbours within $r_{\max}$. These are stored as a sorted adjacency list:

$$\mathcal{N}(i) = \{j_1, j_2, \dots, j_{m_i}\}, \quad j_1 < j_2 < \dots < j_{m_i}. \quad (1)$$

Alongside each neighbour index, the radial-bin index (an integer byte) and optionally a direction-pixel byte (for the parity 4PCF) are stored. The KD-tree and raw coordinates are then discarded; all subsequent NPCF queries operate entirely on the adjacency lists.

The total memory footprint is $O(Nm)$ rather than $O(N^2)$. The sorted order in Equation (1) is a key invariant exploited by the new merge-walk kernel.

2.2. Triangle and tetrahedron enumeration

For the 3PCF, GRAMSCI uses a hub-and-spoke loop: for each hub i and each neighbour $j \in \mathcal{N}(i)$, it checks whether a second neighbour $k \in \mathcal{N}(i)$ with $k > j$ is also in $\mathcal{N}(j)$. If so, the triangle (i, j, k) is counted in the appropriate (r_{ij}, r_{ik}, r_{jk}) bin. The 4PCF extends this to tetrahedra (i, j_1, j_2, j_3) where every pair is connected. The original membership test was a binary search on $\mathcal{N}(j)$, costing $O(\log m)$ per candidate.

3. NEW CONTRIBUTIONS

3.1. Merge-walk: $O(m)$ sorted-set intersection

Since the hub tail $\{k \in \mathcal{N}(i) : k > j\}$ and $\mathcal{N}(j)$ are both sorted by node index (Equation 1), their intersection can be found in a single linear pass with two pointers a and b :

- a walks the hub tail, starting at the entry after j .
- b walks $\mathcal{N}(j)$ from its beginning.

At each step:

- $\mathcal{N}(i)[a] = \mathcal{N}(j)[b]$: triangle found; accumulate the (r_{ij}, r_{ik}, r_{jk}) bin contribution; advance both.

- $\mathcal{N}(i)[a] < \mathcal{N}(j)[b]$: advance a .
- otherwise: advance b .

Each list is visited at most once, giving $O(m_i + m_j)$ cost per spoke — an $O(\log m)$ improvement over binary search. A schematic Python implementation is shown in Listing 1.

Listing 1. Merge-walk for the 3PCF.

```
for i in hubs:
    for k1, id1 in enumerate(neighbours[i]):
        a = k1 + 1 # hub tail: IDs > id1
        b = 0      # id1's full list
        while a < len(neighbours[i]) and \
              b < len(neighbours[id1]):
            if neighbours[i][a] == neighbours[id1][b]:
                accumulate(bins_i[k1],
                           bins_i[a],
                           bins_id1[b])
                a += 1; b += 1
            elif neighbours[i][a] < neighbours[id1][b]:
                a += 1
            else:
                b += 1
```

3.1.1. Three-way merge for the 4PCF

For the 4PCF, once the k_2 walk has located a second spoke j_2 , the k_3 loop must find a third spoke j_3 that appears simultaneously in the hub tail beyond j_2 , in $\mathcal{N}(j_1)$, and in $\mathcal{N}(j_2)$. We generalise the two-pointer walk to three pointers α, β, γ :

- α : hub tail beyond j_2 .
- β : $\mathcal{N}(j_1)$, starting at the position just after j_2 (reused from the k_2 walk, so no rescanning).
- γ : $\mathcal{N}(j_2)$, pre-advanced past j_2 's own index (entries $\leq j_2$ can never match the hub tail).

Let $h_\alpha, h_\beta, h_\gamma$ be the current candidate node indices. If all three are equal, we record the tetrahedron and advance all three. Otherwise every pointer at the current minimum is advanced:

$$\begin{aligned} h_\alpha \leq h_\beta \text{ and } h_\alpha \leq h_\gamma : & \quad \alpha \leftarrow \alpha + 1 \\ h_\beta \leq h_\alpha \text{ and } h_\beta \leq h_\gamma : & \quad \beta \leftarrow \beta + 1 \\ h_\gamma \leq h_\alpha \text{ and } h_\gamma \leq h_\beta : & \quad \gamma \leftarrow \gamma + 1 \end{aligned} \quad (2)$$

(Multiple conditions can be true simultaneously when two minima are tied; in that case multiple pointers are advanced in the same step.) This visits each list at most once, giving $O(m)$ cost per (i, j_1, j_2) triple compared with two $O(\log m)$ binary searches in the original code.

3.2. Parity-decomposed 4PCF

A standard 4PCF estimator gives one number per tetrahedron configuration. GRAMSCI v2 adds a parity decomposition that separates the signal into parity-even (ζ^+) and parity-odd (ζ^-) channels. For each tetrahedron (i, j_1, j_2, j_3) , the handedness is determined by the signed volume

$$V = (\mathbf{r}_{ij_1} \times \mathbf{r}_{ij_2}) \cdot \mathbf{r}_{ij_3}. \quad (3)$$

The direction of each spoke is also quantised into a pixel on the unit sphere (stored as a single byte per edge at graph-construction time with $\sim 17\%$ memory overhead), enabling efficient grouping by parity at query time without retaining the original coordinates.

The parity-odd 4PCF ζ^- vanishes for any statistically parity-symmetric field; a non-zero measurement signals parity violation. Searches for such a signal in BOSS galaxy catalogues (Hou et al. 2023) have generated significant interest, and independent configuration-space measurements provide a direct cross-check on spherical-harmonic-based analyses.

3.3. Python interface

A lightweight Python package (requiring only NumPy) wraps the compiled Fortran binary. Input/output is handled automatically through a temporary directory; the user supplies NumPy arrays and receives structured result objects:

Listing 2. Minimal Python usage.

```
import gramsci, numpy as np

# pos: (N,3) float64, weights: (N,) float64
r3 = gramsci.compute_3pcf(
    pos_data, w_data, pos_rand, w_rand,
    rmin=1.0, rmax=30.0, nbins=6)
print(r3.zeta) # shape (n_configs,)
print(r3.NNN, r3.r1, r3.r2, r3.r3)

r4 = gramsci.compute_4pcf(
    pos_data, w_data, pos_rand, w_rand,
    rmin=1.0, rmax=30.0, nbins=3,
    parity=True)
print(r4.zeta_even) # parity-even 4PCF
print(r4.zeta_odd) # parity-odd 4PCF
```

The package is installable with `pip install -e .` from the `python/` subdirectory of the repository.

4. PERFORMANCE BENCHMARKS

4.1. Test catalogue

All benchmarks used a Gaussian mock with $N_g = 207\,246$ galaxies and $N_r = 500\,000$ randoms in a periodic box. Timings are wall-clock seconds on a single node with 4 OpenMP threads, divided by thread count

Table 1. Wall-clock speedup of the merge-walk over binary search.

Kernel	$r_{\max}$ [$h^{-1}\text{Mpc}$]	Bsearch [s]	Merge-walk [s]	Speedup
3PCF	30	6.6	4.5	$1.46\times$
3PCF	40	34	21	$1.65\times$
3PCF	55	223	109	$2.04\times$
4PCF	30	61.6	18.8	$3.27\times$
4PCF	40	666.7	174.1	$3.83\times$
4PCF (parity)	30	69.2	24.8	$2.79\times$

NOTE—Single-node, 4 OpenMP threads; times divided by thread count. The 4PCF gain is larger than the 3PCF gain because the original code had two binary searches in the innermost loop.

to give effective single-core seconds for fair comparison. Radial bins are equally spaced from $r_{\min} = 1\,h^{-1}\text{Mpc}$ to $r_{\max}$.

4.2. Speedup results

Table 1 summarises the results. The speedup grows with $r_{\max}$ because the mean neighbour count $m \propto r_{\max}^3$ and the savings per inner-loop step scale as $\log_2 m$. At $r_{\max} = 55\,h^{-1}\text{Mpc}$ ($m \approx 700$) the 3PCF achieves $2\times$; at $r_{\max} = 40\,h^{-1}\text{Mpc}$ the 4PCF achieves $3.8\times$ because two independent binary searches were replaced by a single three-pointer walk.

4.3. Correctness verification

Both implementations are run on the same catalogue and compared bin by bin. The maximum absolute difference is $|\Delta\zeta| < 10^{-27}$ for the 3PCF and 4PCF — consistent with floating-point non-associativity from different OpenMP thread accumulation order, not algorithmic discrepancy. The regression test suite (included in the repository) checks isolated-pair 2PCF, regular-tetrahedra connected 4PCF, and chiral-tetrahedra parity 4PCF against known analytic values.

5. SCIENTIFIC APPLICATIONS

Primordial non-Gaussianity.—The connected 4PCF ζ_{conn} is a direct configuration-space probe of the primordial trispectrum. The $3.8\times$ speedup at $r_{\max} = 40\,h^{-1}\text{Mpc}$ makes it feasible to evaluate ζ_{conn} across the full BAO range for DESI-scale catalogues, where covariance estimation requires $\mathcal{O}(10^3)$ mock realisations.

Parity violation.—Hou et al. (2023) reported a non-zero parity-odd 4PCF in BOSS DR12 using a spherical-

harmonic estimator. The GRAMSCI configuration-space parity estimator provides a complementary cross-check that is sensitive to the full angular structure of the signal without assuming a harmonic decomposition.

BAO with higher-order statistics.—GRAMSCI supports anisotropic 3PCF and 4PCF (via μ binning for redshift-space distortions), enabling joint 2PCF+3PCF+4PCF BAO fits that extract additional signal-to-noise from the same catalogue.

6. SUMMARY

We have presented three additions to GRAMSCI:

1. **Merge-walk inner kernel.** Replacing binary-search with two-pointer sorted-set intersection reduces the 3PCF inner loop from $O(m \log m)$ to $O(m)$ and the 4PCF inner loop — which required two binary searches — correspondingly further. Measured speedups are $1.5\text{--}2\times$ for the 3PCF and $2.8\text{--}3.8\times$ for the 4PCF at survey-relevant scales. The key prerequisite — adjacency lists sorted by

node index — was already guaranteed by the KD-tree construction and required no data-structure change.

2. **Parity-decomposed 4PCF.** A direction-pixel byte stored per edge at graph-construction time allows the 4PCF to be decomposed into parity-even and parity-odd channels at negligible additional cost, enabling direct tests of parity violation.
3. **Python interface.** A thin NumPy-based wrapper calls the Fortran engine via subprocess, handling all file I/O automatically and returning structured result objects.

The code is publicly available at <https://github.com/csabiu/Gramsci>.

Software. GRAMSCI (Sabiou et al. 2019); KDTREE2 (Kennel 2004); NumPy (Harris et al. 2020).

¹ [To be completed.]

REFERENCES

- Anderson, L., et al. 2014, MNRAS, 441, 24.
<https://arxiv.org/abs/1312.4877>
- Gaztañaga, E., & Scoccimarro, R. 2005, MNRAS, 361, 824.
<https://arxiv.org/abs/astro-ph/0501637>
- Harris, C. R., et al. 2020, Nature, 585, 357
- Hou, J., Cahn, R. N., Philcox, O. H., & Slepian, Z. 2023, JCAP, 2023, 045. <https://arxiv.org/abs/2206.14083>
- Jarvis, M., Bernstein, G., & Jain, B. 2004, MNRAS, 352, 338. <https://arxiv.org/abs/astro-ph/0307393>
- Kennel, M. B. 2004, KDTREE2: Fortran 95 and C++ software for efficient nearest-neighbour search.
<https://arxiv.org/abs/physics/0408067>
- Philcox, O. H., & Slepian, Z. 2022, MNRAS, 509, 2457.
<https://arxiv.org/abs/2105.08722>
- Sabiou, C. G., et al. 2019, ApJS, 242, 16.
<https://arxiv.org/abs/1901.00296>
- Sefusatti, E., Crocce, M., Pueblas, S., & Scoccimarro, R. 2006, Phys. Rev. D, 74, 023522.
<https://arxiv.org/abs/astro-ph/0512406>
- Sefusatti, E., & Komatsu, E. 2007, Phys. Rev. D, 76, 083004. <https://arxiv.org/abs/0705.0343>
- Sinha, M., & Garrison, L. H. 2020, MNRAS, 491, 3022.
<https://arxiv.org/abs/1911.03545>